Material del Curso

2. Tipos, Limpieza y Transformacion

Clase 2: Tipos de Datos, Limpieza y Transformación de Datos

1. Introducción

Objetivo de la Clase

El éxito de cualquier proyecto de minería de datos depende en gran medida de la calidad de los datos utilizados. En esta clase, nos enfocaremos en la importancia de la limpieza y preprocesamiento de datos para garantizar análisis precisos y modelos eficientes.

Al finalizar esta clase, deberás ser capaz de:

- Identificar y clasificar tipos de datos en distintos conjuntos de datos.
- Comprender la importancia de la limpieza de datos y aplicar técnicas para detectar valores faltantes, inconsistencias y valores atípicos.
- Realizar transformaciones en los datos para mejorar su interpretación y su uso en modelos de minería de datos.
- Aplicar herramientas de análisis exploratorio en Python para evaluar la calidad de un conjunto de datos.

1.1. Evolución Histórica y Desafíos del Volumen de Datos

En las últimas décadas, el crecimiento exponencial de los datos ha cambiado la forma en que las organizaciones gestionan la información.

Evolución de los datos en la historia:

1. Bases de datos tradicionales (años 70-90): Los sistemas almacenaban información en bases de datos relacionales estructuradas (SQL).

2. Big Data y la explosión digital (2000-presente): El auge de Internet, redes sociales, sensores IoT y la digitalización de documentos generaron volúmenes masivos de datos estructurados y no estructurados.

3. Minería de Datos y Machine Learning: La disponibilidad de datos llevó al desarrollo de herramientas avanzadas para extraer conocimiento mediante algoritmos y modelos predictivos.

Desafíos actuales en la gestión de datos:

- Volumen: Manejar cantidades masivas de datos (Big Data).
- Variedad: Integración de datos estructurados (tablas SQL) con datos no estructurados (imágenes, audios, redes sociales).
- Veracidad: Detección de datos incorrectos, sesgados o duplicados.
- Velocidad: Procesamiento de datos en tiempo real (ejemplo: detección de fraudes bancarios).

Ejemplo Real:

Un hospital puede tener datos de pacientes en bases SQL, imágenes médicas en DICOM, registros de voz en notas médicas y documentos en PDF. Para analizarlos, es necesario aplicar técnicas de preprocesamiento específicas.

1.2. Importancia del Preprocesamiento de Datos

Antes de aplicar algoritmos de minería de datos o Machine Learning, es fundamental preparar los datos. Un conjunto de datos sin preprocesar puede contener:

- Valores faltantes → Información incompleta.
- Valores atípicos (outliers) → Datos extremos que pueden sesgar el análisis.
- Valores inconsistentes → Incoherencias en nombres, formatos o unidades.
- Duplicados → Registros repetidos que afectan la precisión de los modelos.

Ejemplo de Problemas en Datos

ID Nombre Edad Fecha de Registro Ingresos Estado

- -------- ------ ---------- ---------- ----------

1 Juan 32 10/05/2023 45000.0 Activo

2 María -28 15-06-2023 55000.0 ACTIVO

3 Pedro 40 20/02/2023 None Inactivo

4 Juan 32 10/05/2023 45000.0 Activo

Problemas detectados:

- Edad negativa (error en la entrada de datos).
- Formato inconsistente en fechas.
- Valor faltante en ingresos.
- Estado inconsistente (“Activo” vs. “ACTIVO”).

- Registro duplicado (Juan aparece dos veces).

Conclusión: Antes de cualquier análisis, debemos limpiar y transformar los datos para evitar sesgos o errores en los modelos.

1.3. Flujo de Trabajo en Minería de Datos

El proceso de minería de datos sigue un flujo estructurado para asegurar la calidad de los datos. Dos enfoques comunes son KDD (Knowledge Discovery in Databases) y CRISP-DM (Cross Industry Standard Process for Data Mining).

Fases del Proceso KDD

1. Selección de datos → ¿Qué datos son relevantes para el análisis?

2. Preprocesamiento → Limpieza de datos, eliminación de duplicados, corrección de errores.

3. Transformación → Normalización, codificación de variables categóricas, generación de nuevas variables.

4. Minería de Datos → Aplicación de modelos de clasificación, regresión o clustering.

5. Evaluación e Interpretación → Validación de los resultados obtenidos.

Modelo CRISP-DM

Un enfoque más estructurado utilizado en la industria que organiza el análisis en seis fases:

1. Comprensión del negocio → Definir el problema y los objetivos del análisis.

2. Comprensión de los datos → Exploración inicial, identificación de errores.

3. Preparación de los datos → Limpieza, transformación y selección de variables.

4. Modelado → Aplicación de algoritmos de minería de datos.

5. Evaluación → Validación del modelo y ajuste de parámetros.

6. Despliegue → Implementación del modelo en producción.

Ejemplo CRISP-DM aplicado a un Banco

- Comprensión del negocio: Reducir la tasa de morosidad en préstamos.
- Comprensión de los datos: Analizar historiales de pago, ingresos y edad de clientes.
- Preparación: Normalizar ingresos, codificar estados de cuenta (Pagado/Moroso).
- Modelado: Usar algoritmos de clasificación para predecir clientes de riesgo.
- Evaluación: Validar con datos de prueba y ajustar el modelo.
- Despliegue: Implementar en el sistema del banco para evaluar nuevos clientes.

Ejercicio para la Sección 1

Objetivo: Comprender la importancia del pre procesamiento y clasificación de datos.

Ejercicio 1: Identificación y Diagnóstico de Problemas en Datos

Dado el siguiente conjunto de datos de clientes de un banco:

Cliente Edad Salario Estado Civil Crédito Aprobado

- ----- ------ --------- ---------- ----------

1 25 50000 Soltero Sí

2 -40 60000 Casado No

3 32 70000 Divorciado Sí

4 22 None Soltero No

5 35 80000 Viudo Sí

Instrucciones:

1. Identifica los problemas en los datos.

2. Propón una estrategia de limpieza.

3. Escribe un código en Python para detectar valores faltantes, atípicos y duplicados.

Ejemplo de Código en Python

```
import pandas as pd

# Crear DataFrame

datos = {
"Cliente": [1, 2, 3, 4, 5],
"Edad": [25, -40, 32, 22, 35],
"Salario": [50000, 60000, 70000, None, 80000],
"Estado_Civil": ["Soltero", "Casado", "Divorciado", "Soltero", "Viudo"],

"Credito_Aprobado": ["Sí", "No", "Sí", "No", "Sí"]
}
df = pd.DataFrame(datos)
# Ver problemas en los datos

print(df.isnull().sum()) # Valores faltantes

print(df[df["Edad"] < 0]) # Valores atípicos
```

2. Tipos de Datos y su Clasificación

La identificación y clasificación de los datos es un paso clave en minería de datos, ya que determina qué técnicas de limpieza y transformación pueden aplicarse.

2.1. Tipos de Datos según su Estructura

Los datos pueden organizarse de diferentes maneras según su formato y origen.

Tipo de Datos Descripción Ejemplo

- --- --- ---

Estructurados Datos organizados en filas y columnas. Se almacenan en bases de datos relacionales. Tabla de clientes con nombres, edades y saldos bancarios.

No estructurados Datos sin formato predefinido, difíciles de almacenar en bases SQL. Imágenes, videos, audios, correos electrónicos.

Semiestructurados Datos que tienen etiquetas o metadatos que facilitan su organización. Archivos JSON, XML, documentos HTML.

Ejemplo en Python – Cargando diferentes tipos de datos

```
import pandas as pd

# Cargar datos estructurados (desde CSV)
df_csv = pd.read_csv("clientes.csv")
# Cargar datos semiestructurados (JSON)
df_json = pd.read_json("clientes.json")
# Inspeccionar los datos

print(df_csv.head())

print(df_json.head())
```

Conclusión: Dependiendo del tipo de dato, se requieren herramientas específicas para su almacenamiento y procesamiento.

2.2. Tipos de Datos según su Naturaleza

Los datos también pueden clasificarse según su naturaleza matemática y uso en modelos analíticos.

Tipo de Dato Descripción Ejemplo

- --- --- ---

Numéricos Valores que representan cantidades medibles. Precio, peso, temperatura.

Categóricos Representan etiquetas o nombres de categorías. Género (Masculino/Femenino), Estado Civil.

Ordinales Categorías con un orden lógico, pero sin una distancia uniforme entre ellas. Nivel educativo (Primaria, Secundaria, Universitaria).

Ejemplo en Python – Identificación de Tipos de Datos en un DataFrame

```
import pandas as pd

# Crear un DataFrame con distintos tipos de datos
df = pd.DataFrame({
"Edad": [25, 30, 35, 40, 45],
"Ingreso": [50000, 60000, 55000, 75000, 90000],
"Estado_Civil": ["Soltero", "Casado", "Viudo", "Casado", "Soltero"],

"Nivel_Educativo": ["Secundaria", "Universitaria", "Secundaria", "Primaria", "Universitaria"]
})
# Inspeccionar los tipos de datos

print(df.dtypes)
```

Salida esperada:

```
Edad int64
Ingreso int64
Estado_Civil object
Nivel_Educativo object
dtype: object
```

Conclusión:

- Los datos numéricos pueden usarse directamente en modelos de predicción.
- Los datos categóricos deben transformarse antes de usarse en algoritmos de Machine Learning.

2.3. Transformación de Datos Categóricos

Algunos algoritmos no pueden trabajar con datos categóricos, por lo que es necesario convertirlos en datos numéricos.

2.3.1. Codificación Ordinal

Usada cuando las categorías tienen un orden específico.

Ejemplo – Convertir Nivel Educativo en Números

```
from sklearn.preprocessing import OrdinalEncoder

# Definir orden de categorías

orden_educativo = [["Primaria", "Secundaria", "Universitaria"]]

# Aplicar codificación ordinal
encoder = OrdinalEncoder(categories=orden_educativo)
df["Nivel_Educativo_Cod"] = encoder.fit_transform(df[["Nivel_Educativo"]])
print(df[["Nivel_Educativo", "Nivel_Educativo_Cod"]])
```

Salida esperada:

```
Nivel_Educativo Nivel_Educativo_Cod
0 Secundaria 1.0
1 Universitaria 2.0
2 Secundaria 1.0
3 Primaria 0.0
4 Universitaria 2.0
```

Útil para datos como: tamaño de ropa (S, M, L), satisfacción del cliente (Bajo, Medio, Alto).

2.3.2. Codificación One-Hot Encoding

Se usa cuando las categorías no tienen un orden jerárquico.

Ejemplo – Convertir Estado Civil en Variables Binarias

```
from sklearn.preprocessing import OneHotEncoder

# Aplicar One-Hot Encoding
encoder = OneHotEncoder(sparse=False, drop="first")
encoded_data = encoder.fit_transform(df[["Estado_Civil"]])
# Convertir a DataFrame
df_encoded = pd.DataFrame(encoded_data, columns=encoder.get_feature_names_out(["Estado_Civil"]))
df = pd.concat([df, df_encoded], axis=1)
print(df.head())
```

Salida esperada:

```
Estado_Civil Estado_Civil_Casado Estado_Civil_Viudo
0 Soltero 0 0
1 Casado 1 0
2 Viudo 0 1
3 Casado 1 0
4 Soltero 0 0
```

Útil para datos como: País, Tipo de Producto, Categoría de Cliente.[]

Ejercicio para la Sección 2

Objetivo: Aplicar la clasificación y transformación de datos categóricos.

Ejercicio 1: Clasificación y Transformación de Datos

Supongamos que tenemos el siguiente conjunto de datos:

ID Nombre Edad Categoría Cliente Estado Civil

- -------- ------ ---------- ----------

1 Juan 32 Oro Soltero

2 María 28 Plata Casado

3 Pedro 40 Platino Viudo

4 Ana 35 Oro Casado

Instrucciones:

1. Clasifica cada columna según su tipo (numérico, categórico, ordinal).

2. Indica qué método de transformación aplicarías para convertir “Categoría Cliente” y “Estado Civil” en datos numéricos.

3. Escribe un código en Python para cargar estos datos en un DataFrame y aplicar la transformación adecuada.

Pista para la transformación en Python

```
import pandas as pd

from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder

# Crear DataFrame

data = {
"ID": [1, 2, 3, 4],
"Nombre": ["Juan", "María", "Pedro", "Ana"],
"Edad": [32, 28, 40, 35],
"Categoria_Cliente": ["Oro", "Plata", "Platino", "Oro"],

"Estado_Civil": ["Soltero", "Casado", "Viudo", "Casado"]
}
df = pd.DataFrame(data)
# Codificación ordinal para "Categoría Cliente"

orden_categoria = [["Plata", "Oro", "Platino"]]
encoder_ordinal = OrdinalEncoder(categories=orden_categoria)
df["Categoria_Cod"] = encoder_ordinal.fit_transform(df[["Categoria_Cliente"]])
# One-Hot Encoding para "Estado Civil"
encoder_onehot = OneHotEncoder(sparse=False, drop="first")
encoded_estado = encoder_onehot.fit_transform(df[["Estado_Civil"]])
df_encoded = pd.DataFrame(encoded_estado, columns=encoder_onehot.get_feature_names_out(["Estado_Civil"]))
df = pd.concat([df, df_encoded], axis=1)
print(df.head())
```

Sección 4: Introducción a la Transformación y Análisis Exploratorio de Datos

Objetivo: Entender qué es la transformación de datos y por qué es fundamental en minería de datos, además de realizar un primer acercamiento al análisis exploratorio.

4.1. ¿Qué es la Transformación de Datos?

Después de limpiar los datos, en muchos casos es necesario modificarlos para mejorar su calidad y usabilidad. Esto se conoce como transformación de datos y es un paso clave en minería de datos y aprendizaje automático.

Ejemplos comunes de transformación:

Conversión de tipos de datos (de texto a numérico, de fecha a componentes de tiempo).

Normalización y escalado (para ajustar valores numéricos a una escala común).

Codificación de variables categóricas (convertir texto en valores numéricos).

Creación de nuevas variables a partir de variables existentes (ejemplo: calcular el IMC a partir de peso y altura).

Ejemplo en Python – Conversión de Datos de Texto a Números

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

# Datos originales
df = pd.DataFrame({
"Cliente": ["A", "B", "C", "D"],

"Estado_Cuenta": ["Pagado", "Moroso", "Pagado", "Moroso"]
})
# Convertir texto a números
encoder = LabelEncoder()
df["Estado_Cuenta_Num"] = encoder.fit_transform(df["Estado_Cuenta"])
print(df)
```

Salida esperada:

```
Cliente Estado_Cuenta Estado_Cuenta_Num
0 A Pagado 1
1 B Moroso 0
2 C Pagado 1
3 D Moroso 0
```

Conclusión: Transformar variables categóricas es clave para usarlas en modelos predictivos.

4.2. ¿Qué es el Análisis Exploratorio de Datos (EDA)?

El Análisis Exploratorio de Datos (Exploratory Data Analysis, EDA) es el proceso de investigar un conjunto de datos mediante estadísticas descriptivas y visualizaciones antes de aplicar modelos de minería de datos.

Objetivos del EDA:

- Identificar patrones generales en los datos.
- Detectar valores atípicos o problemas en los datos.
- Visualizar distribuciones y correlaciones.

Ejemplo en Python – Análisis Rápido con Pandas

```
import pandas as pd

# Cargar datos
df = pd.DataFrame({
"Edad": [25, 32, 40, 35, 50],
"Ingresos": [30000, 45000, 60000, 55000, 80000]
})
# Resumen estadístico

print(df.describe())

# Correlación entre variables

print(df.corr())
```

Salida esperada: Estadísticas descriptivas como media, mediana, percentiles y correlaciones entre variables.

Conclusión: El EDA ayuda a entender la estructura de los datos antes de aplicar modelos.

Ejercicio para la Sección 4

Objetivo: Aplicar conceptos básicos de transformación y análisis exploratorio.

Ejercicio 1: Exploración y Transformación de Datos

Dado el siguiente conjunto de datos:

ID Nombre Edad Categoría Cliente Ingresos

- -------- ------ ---------- ----------

1 Juan 32 Oro 45000

2 María 28 Plata 55000

3 Pedro 40 Platino 70000

4 Ana 35 Oro 60000

Instrucciones:

1. Calcula estadísticas descriptivas (media, mediana, valores máximos y mínimos).

2. Convierte la columna "Categoría Cliente" en una variable numérica.

3. Calcula la correlación entre Edad e Ingresos.

Pista para la solución en Python

```
import pandas as pd

from sklearn.preprocessing import OrdinalEncoder

# Crear DataFrame
df = pd.DataFrame({
"ID": [1, 2, 3, 4],
"Nombre": ["Juan", "María", "Pedro", "Ana"],
"Edad": [32, 28, 40, 35],
"Categoria_Cliente": ["Oro", "Plata", "Platino", "Oro"],
"Ingresos": [45000, 55000, 70000, 60000]
})
# Estadísticas descriptivas

print(df.describe())

# Codificación ordinal de "Categoría Cliente"

orden_categoria = [["Plata", "Oro", "Platino"]]
encoder = OrdinalEncoder(categories=orden_categoria)
df["Categoria_Cod"] = encoder.fit_transform(df[["Categoria_Cliente"]])
# Calcular correlación

print(df[["Edad", "Ingresos"]].corr())

print(df)
```

Instancia de evaluación — Actividad Obligatoria (Clase 2)

Título: Limpieza, Transformación y Análisis Exploratorio de Datos con LLMs Objetivo: Aplicar identificación de tipos de datos, limpieza, transformación y análisis exploratorio, integrando el uso de un LLM (p. ej., ChatGPT o Gemini) como apoyo metodológico.

Entrega en Moodle

- Subir dos archivos:
- AO1_Clase2_Apellido_Nombre.ipynb (el notebook debe ejecutar inicio a fin sin errores; "Restart & Run All").
- AO1_Clase2_Apellido_Nombre.pdf (exportación del mismo notebook, con todas las salidas visibles).
- Incluir en el notebook:
- Clasificación de tipos de datos y criterios de manejo.
- Limpieza (faltantes, consistencias en categóricas, outliers en Salario).
- Transformación (codificación One-Hot u ordinal; normalización/estandarización de Salario; "Años hasta jubilación" = 65 − edad).
- EDA con al menos dos visualizaciones y breve interpretación.
- Uso de LLM: 1–2 prompts y síntesis (3–5 líneas por prompt) de lo aportado, con validación propia.
- La fecha y hora límite se rigen por lo indicado en Moodle.

Criterio de Evaluación.

Mediante la siguiente rúbrica en la plataforma moodle:

Criterio	No logrado. 0 ptos	Parcialmente logrado. 1 pto	Completamente logrado. 2 puntos.
1. Tipos de datos y criterio de manejo	Clasificación incorrecta o incompleta; sin justificación del manejo.	Clasificación mayormente correcta con omisiones; justificación general.	Clasificación completa y correcta; justifica el manejo y su impacto en el flujo de trabajo.
2. Limpieza (faltantes, inconsistencias, outliers en Salario)	Faltantes sin tratamiento; categóricas inconsistentes; outliers no detectados o mal tratados.	Trata faltantes y categóricas con resultados parciales; detecta outliers con justificación limitada.	Trata faltantes con criterio; unifica categóricas; detecta y trata outliers con método justificado (IQR/z-score/percentiles) e informa umbrales.
3. Transformación (encoding, normalización, variable nueva)	Codificación inadecuada o ausente; normalización ausente; "Años hasta jubilación" incorrecta o ausente.	Codificación y normalización con errores menores o sin justificación; variable creada con detalles a corregir.	Codificación apropiada (One-Hot u ordinal) y normalización/estandarización justificadas; "Años hasta jubilación" correcta; decisiones explicadas.
4. EDA (gráficas e interpretación)	Menos de 2 visualizaciones o irrelevantes; sin interpretación.	≥ 2 visualizaciones pertinentes; interpretación básica.	≥ 2 visualizaciones pertinentes con interpretación clara de tendencias, dispersiones y relaciones; hallazgos fundamentados.
5. Claridad, reproducibilidad y uso de LLM	Notebook no ejecuta de inicio a fin; sin evidencia útil de LLM; documentación deficiente.	Notebook ejecuta con fricciones menores; prompts incluidos con integración limitada; documentación suficiente.	Notebook ejecuta completamente; decisiones documentadas; prompts y síntesis del LLM integrados con validación propia; nomenclatura de archivos correcta.

Situación problemática.

Contexto del Problema

Una empresa de recursos humanos quiere analizar un conjunto de datos de empleados para identificar patrones en sueldos, edades y categorías laborales.

El dataset contiene algunos problemas comunes, como valores faltantes, datos inconsistentes y variables categóricas sin codificar. Además, la empresa quiere entender la relación entre edad, salario y nivel educativo.

Tareas a Realizar

1. Identificación de Tipos de Datos

Clasifica cada columna según su tipo (numérico, categórico, ordinal, fecha) y explica cómo se debería manejar cada una.

2. Limpieza de Datos

- Detecta y corrige valores faltantes en las columnas necesarias.
- Identifica y elimina valores atípicos en la columna "Salario".
- Corrige inconsistencias en los datos categóricos (ejemplo: "Licenciado" y "licenciado" deben ser tratados como el mismo valor).

3. Transformación de Datos

- Convierte las variables categóricas en numéricas utilizando codificación ordinal o One-Hot Encoding.
- Aplica normalización al salario para facilitar comparaciones.
- Crea una nueva variable "Años hasta jubilación" suponiendo que la edad de retiro es 65 años.

4. Análisis Exploratorio con LLM

Usa ChatGPT o una herramienta LLM para ayudarte en alguna de estas tareas:

- Generar un resumen de los datos.
- Explicar cómo manejar valores faltantes.
- Sugerir formas de visualizar los datos.

Ejemplo de consulta a ChatGPT:

"Tengo un dataset con sueldos y edades, ¿cómo puedo detectar y eliminar outliers en Python?"

Comparte en el foro la consulta que hiciste y la respuesta que te dio el LLM.

Dataset para el Ejercicio

ID Nombre Edad Nivel Educativo Salario Categoría Estado

- -------- ------ ---------- ---------- ---------- ----------

1 Juan 32 Licenciado 50000.0 Junior Activo

2 María -28 Ingeniero 60000.0 Senior ACTIVO

3 Pedro 40 Doctorado None Senior Inactivo

4	Ana	35	Licenciado	55000.0	Junior	Activo
5	Luis	50	Doctorado	200000.0	Manager	Inactivo

Pistas para Resolverlo en Python

Código base para comenzar el análisis:

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler, OrdinalEncoder

# Cargar los datos
df = pd.DataFrame({
"ID": [1, 2, 3, 4, 5],
"Nombre": ["Juan", "María", "Pedro", "Ana", "Luis"],
"Edad": [32, -28, 40, 35, 50],
"Nivel_Educativo": ["Licenciado", "Ingeniero", "Doctorado", "Licenciado", "Doctorado"],
"Salario": [50000, 60000, None, 55000, 200000],
"Categoría": ["Junior", "Senior", "Senior", "Junior", "Manager"],

"Estado": ["Activo", "ACTIVO", "Inactivo", "Activo", "Inactivo"]
})
# 1 Corrección de edad negativa
df["Edad"] = df["Edad"].abs()
# 2 Imputación del salario faltante con la mediana
df["Salario"].fillna(df["Salario"].median(), inplace=True)
# 3 Detectar y eliminar valores atípicos con IQR
Q1, Q3 = df["Salario"].quantile([0.25, 0.75])
IQR = Q3 - Q1
df = df[~((df["Salario"] < (Q1 - 1.5 * IQR)) | (df["Salario"] > (Q3 + 1.5 * IQR)))]
# 4 Codificación ordinal para "Nivel Educativo"
encoder_ordinal = OrdinalEncoder(categories=[["Licenciado", "Ingeniero", "Doctorado"]])
df["Nivel_Cod"] = encoder_ordinal.fit_transform(df[["Nivel_Educativo"]])
# 5 Normalización del salario
scaler = MinMaxScaler()
df["Salario_Normalizado"] = scaler.fit_transform(df[["Salario"]])
# 6 Creación de nueva variable "Años hasta jubilación"
df["Años_Hasta_Jubilación"] = 65 - df["Edad"]
print(df)
```

Bibliografía

1. **Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques (3rd ed.). Elsevier. ISBN: 978-0-12-381479-1.**
2. **Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2017). Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann Publishers.**
3. **Hernández Orallo, J., Ramírez Quintana, M. J., & Ferri Ramírez, C. (2004). Introducción a la Minería de Datos. Pearson-Prentice Hall.**
4. **Provost, F., & Fawcett, T. (2013). Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking. O'Reilly Media.**
5. **Python Software Foundation. (n.d.). Welcome to Python.org. Recuperado el [fecha de consulta], de https://www.python.org/**
6. **The pandas development team. (n.d.). Pandas: Powerful Python Data Analysis Toolkit. Recuperado el [fecha de consulta], de https://pandas.pydata.org/**
7. **Seaborn Development Team. (n.d.). Seaborn: Statistical Data Visualization. Recuperado el [fecha de consulta], de https://seaborn.pydata.org/**
8. **Project Jupyter. (n.d.). Project Jupyter. Recuperado el [fecha de consulta], de https://jupyter.org/**
9. **Bengfort, B., Bilbro, R., & Ojeda, T. (2018). Applied Text Analysis with Python: Enabling Language-Aware Data Products with Machine Learning. O'Reilly Media.**